Prune-and-Merge: Training-Free Scaling of Vision Transformers*

Anonymous CVPR submission

Paper ID *****

Abstract

Vision Transformers achieve strong performance but incur high computational costs due to quadratic token interactions. Training-free token reduction methods improve efficiency, but robustness of these methods remains underexplored. In this study, we propose Prune-and-Merge ViT, which is a training-free framework that can be applied directly to off-the-shelf models. We investigated the robustness of token reduction in pretrained Vision Transformers. We combined class-attention-based token pruning with similarity-based token merging to reduce redundancy while preserving semantic information. To evaluate robustness, we introduced two metrics: Relative Accuracy Drop (RAD), which measures relative accuracy degradation under transformations and Prediction Flip Rate (PFR), which quantifies prediction consistency. Experiments on pretrained ViT and DeiT models showed that the proposed Prune-and-Merge reduction framework reduced computation competitively while maintaining robustness and better accuracy than existing token reduction methods. Our analysis suggests that hybrid token reduction does not introduce hallucination-like behavior and preserves prediction stability for common image transformations.

1. Introduction

Vision Transformers (ViTs) represent a paradigm shift in computer vision by treating image data as a sequence of discrete visual tokens, moving away from the traditional inductive biases of convolutional neural networks [3]. In this framework, an input image is partitioned into a grid of non-overlapping patches, which are linearly projected into a high-dimensional embedding space to form N input tokens. To capture global dependencies, these tokens, along with a learnable class ([CLS]) token, are processed through a series of Transformer blocks consisting of multi-head self-attention (MHSA) and feed-forward networks (FFN). While this architecture effectively models long-range spatial relationships, the number of tokens remains constant across all layers, resulting in a computational footprint that scales quadratically ($O(N^2)$) with the sequence length.

To address this issue, several token reduction methods have been proposed, including token pruning and token merging [2, 7]. These approaches aim to remove redundant tokens or aggregate similar tokens to reduce computation. Many recent methods operate in a training-free setting, enabling direct application to pretrained models without retraining [2, 7]. While such approaches improve efficiency, they may introduce instability under input transformations, potentially affecting prediction consistency. In addition, aggressive token reduction at lower keep rates can degrade performance, as fewer tokens remain to capture essential visual information. These observations motivate a systematic analysis of prediction stability under aggressive token reduction.

Existing approaches apply pruning or merging in isolation, which results in semantic loss or hallucination-like errors under input transformations. We address these limitations with *Prune-and-Merge ViT*, a plug-and-play, training-free framework that synergistically combines class-attention-based pruning with similarity-driven token merging for off-the-shelf ViT [3] and DeiT [11] architectures. In the first stage, the model discards low-importance tokens via top-k selection on class attention scores [7], effectively filtering noise while retaining salient features. Subsequently, a merging stage aggregates the remaining redundant tokens through bipartite matching on cosine similarities [2], preserving the semantic manifold within a compact bottleneck representation $\mathbf{T}' \in \mathbb{R}^{N' \times D}$, where $N' \ll N$. Unlike single-strategy methods, our framework provides fine-grained control over the reduction profile. By adjusting the pruning ratio and merging threshold, the token budget can be flexibly controlled while preserving semantic stability.

To systematically evaluate robustness, we introduce two metrics: Relative Accuracy Drop (RAD), which

*Code will be made publicly available upon acceptance.

measures relative accuracy degradation under input transformations and Prediction Flip Rate (PFR), which quantifies prediction consistency between original and transformed inputs. These metrics allow us to analyze whether token reduction introduces hallucination-like behavior or instability.

Experiments on pretrained ViT and DeiT models demonstrate that *Prune-and-Merge ViT* competitively reduces computation while maintaining robustness and achieving strong accuracy. Our analysis further shows that hybrid token reduction preserves prediction stability under common image transformations.

Our contributions are summarized as follows:

- We propose *Prune-and-Merge ViT*, a training-free hybrid token reduction framework for Vision Transformers.
- We introduce RAD and PFR to evaluate robustness and prediction stability under semantics-preserving transformations.
- We analyze the robustness of token reduction and show that hybrid pruning and merging preserve stability while reducing computation.

2. Related Works

2.1. Vision Transformer Foundations and Distillation

The Vision Transformer (ViT) architecture [3] has been further optimized through specialized training strategies. Notably, DeiT [11] introduced a distillation token that interacts with patch embeddings via self-attention to inherit the inductive biases of convolutional 'teacher' models. This work demonstrated that hard distillation can significantly outperform soft distillation and that the interplay between the [CLS] and distillation tokens provides a more robust classifier than either token in isolation. While effective, these models still maintain a fixed sequence length, leading to high inference costs.

2.2. Dynamic and Learned Token Pruning

To reduce computation, several methods focus on discarding redundant tokens. DynamicViT [10] introduced a learned prediction module to hierarchically prune tokens throughout the network. A key finding in learned pruning is that masking tokens at the input layer significantly harms performance, thus, sparsification is typically applied between transformer blocks. While learned modules offer high accuracy, they require additional training and backpropagation. In contrast, EViT [7] and Top-K strategies provide training-free alternatives by ranking token importance based on class-attention scores, identifying and removing 'background' patches that contribute minimally to the final classification.Recent advancements like V-Pruner [12] further refine this by employing globally-informed importance metrics to ensure that locally important but globally redundant features are accurately identified. Recent extensions to state-space models [14] further refine this paradigm, yet hybrids remain underexplored.

2.3. Token Merging and Compression

A more recent paradigm shifts from discarding information to aggregating it. Token Merging (ToMe) [2] utilizes bipartite soft matching to fuse similar tokens based on the dot product of their Key (K) vectors. Unlike pruning which often uses masking that doesn't always translate to hardware speedups merging physically reduces the number of tokens passed to the MLP block, leading to tangible throughput gains. Recent extensions have explored using unequal bipartite graphs to achieve higher compression ratios ($> 50\%$) and applying these techniques to Stable Diffusion to accelerate generative tasks[1]. Furthermore, studies like "Which Tokens to Use?"[5] suggest that ToMe is most effective when keeping at least 50% of the token budget, often outperforming baseline DeiT models by leveraging 'Proportional Attention' to account for the varying weights of merged tokens. Spatial-preserving hybrids like prune-and-merge [8] and adaptive merging [4] enhance locality but lack robustness guarantees, which our framework addresses. Furthermore, hardware-centric frameworks such as VisionTrim [13] have begun optimizing token reduction for specific edge-device latencies, though they often prioritize throughput over prediction stability.

2.4. Plug-and-Play Compression and Transferability

Recent research has addressed the 'gap' between models trained at different compression degrees. ToCom[6] identifies that transferring a model between different compression levels is often inefficient. To mitigate this, it introduces a pretrained 'plug-in' module that models the performance gap between source and target compression degrees, enabling more efficient inference-time scaling without retraining. Extensions to vision-language models [15] further validate this paradigm. Our work builds on this 'plug and play' philosophy by combining pruning and merging into a single, training-free framework that offers more granular control than these monolithic strategies. Finally, as noted by Tian et al. [9], the transferability of these compressed weights to smaller, specialized subsets like ImageNet-100 requires careful alignment to maintain the model's original semantic robustness.

3. Method

3.1. Overview of the Hybrid Framework

We propose *Prune-and-Merge ViT* (see Figure 1), a dual-stage, training-free token reduction framework. Unlike monolithic strategies that either discard or fuse tokens in isolation, our approach treats token reduction as a sequential filtering and compression task. Given an input sequence $\mathbf{x}$ of N tokens, our framework applies a hierarchical reduction at predefined layers $l \in \{3, 6, 9\}$ [5]. This schedule ensures that the model preserves high-resolution spatial information in the early layers before progressively reducing the sequence length in deeper blocks.

The reduction intensity at each stage is governed by a composite keep-rate $K = k_p \cdot k_m$. Here, $k_p \in (0, 1]$ represents the pruning keep-rate, which determines the fraction of tokens retained based on semantic importance; the pruning stage thus acts as a semantic filter. Conversely, $k_m \in (0, 1]$ represents the merging keep-rate, which defines the fraction of tokens remaining after bipartite matching is applied to the pruned subset. This multiplicative approach allows for fine-grained control over the total token budget while ensuring the reduction scales proportionally with the remaining sequence length.

3.2. Stage 1: Class-Attention Pruning

Stage 1 retains semantically important tokens by leveraging the attention weights from the `[CLS]` token, following the strategy of EViT [7]. Given input tokens $\mathbf{x} \in \mathbb{R}^{B \times N \times C}$, multi-head self-attention produces an attention map $\mathbf{A} \in \mathbb{R}^{B \times H \times N \times N}$. We measure patch importance using the attention from `[CLS]` to the $P = N - n_s$ image patch tokens:

$$\mathbf{A}_{\text{cls}} = \mathbf{A}[:, :, 0, n_s :] \in \mathbb{R}^{B \times H \times P}, \tag{1}$$

$$\bar{\mathbf{a}} = \frac{1}{H} \sum_{h=1}^{H} \mathbf{A}_{\text{cls}}^{(h)} \in \mathbb{R}^{B \times P}, \tag{2}$$

where n_s denotes the number of special tokens. The top-k attentive tokens are selected using the pruning keep-rate k_p:

$$k = \max(1, \lceil k_p \cdot P \rceil), \qquad \mathcal{I}_{\text{att}} = \text{TopK}(\bar{\mathbf{a}}, k). \tag{3}$$

To prevent the total loss of background context, we categorize the remaining indices as inattentive tokens $\mathcal{I}_{\text{inatt}}$. We aggregate them into a single representative token $\mathbf{x}_{\text{fused}}$ via attention-weighted averaging:

$$\mathbf{x}_{\text{fused}} = \sum_{i \in \mathcal{I}_{\text{inatt}}} \bar{a}_i \mathbf{x}_i. \tag{4}$$

The resulting sequence $\mathbf{x}_{\text{stage1}} = [\mathbf{x}_{\mathcal{I}_{\text{att}}}, \mathbf{x}_{\text{fused}}] \in \mathbb{R}^{B \times (k+1) \times C}$ concentrates computation on salient regions while preserving global context.

3.3. Stage 2: Similarity-Based Token Merging

Following the pruning and fusion in Stage 1, the resulting sequence $\mathbf{x}_{\text{stage1}}$ (of length $N_p = k + 1$) undergoes bipartite merging inspired by ToMe [2]. We partition the tokens into two disjoint sets, $\mathbb{A}$ and $\mathbb{B}$, of roughly equal size. We identify candidates for merging by computing the cosine similarity between the key features $\mathbf{k}$ of tokens in $\mathbb{A}$ and $\mathbb{B}$, drawing exactly one edge from every token in $\mathbb{A}$ to its most similar counterpart in $\mathbb{B}$.

The merging intensity is governed by the merging keep-rate $k_m \in (0, 1]$, which defines the number of merges r:

$$r = \lfloor (1 - k_m) \cdot N_p \rfloor. \tag{5}$$

By retaining only the r most similar edges, we ensure that only the most redundant features are fused.

Token Size Tracking and Proportional Attention. As tokens are merged, we track a token size vector $\mathbf{s}$, where each entry s_i denotes the number of original patches a token represents [2]. When merging two tokens i and j, we perform a weighted average:

$$\mathbf{x}_{\text{merged}} = \frac{s_i \mathbf{x}_i + s_j \mathbf{x}_j}{s_i + s_j}. \tag{6}$$

To maintain the correct softmax distribution in subsequent layers, we apply proportional attention [2]:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} + \log \mathbf{s} \right). \tag{7}$$

The inclusion of the $\log \mathbf{s}$ term ensures that a merged token exerts the same mathematical influence as if its constituent patches were still present as separate, identical copies. This dual-stage consolidation ensures that the model remains expressive and accurate even under high compression ratios.

4. Experiments

4.1. Implementation Details

To ensure high-fidelity evaluation of the token manifolds, all inference was performed in full precision (FP32). We evaluated our framework across two major model families: the standard Vision Transformer (ViT) and the Data-efficient Image Transformer (DeiT), covering Tiny, Small and Base scales (16×16 patch size, 224×224 resolution).

We utilize official pre-trained weights and evaluate performance on the ImageNet-100 validation set

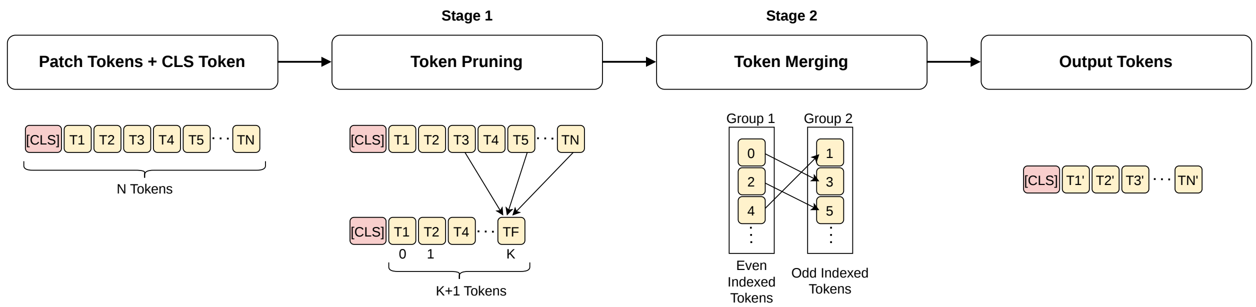

Figure 1. **Overview of the Prune-and-Merge ViT Architecture.** The framework processes tokens through a dual-stage reduction at layers $l \in \{3, 6, 9\}$. Stage 1 (Pruning) filters out background tokens based on class-attention scores k_p, while Stage 2 (Merging) aggregates redundant salient tokens using bipartite matching k_m.

Table 1. Top-1 Accuracy comparison for DeiT and ViT models under different keep rates.

	DeiT-Tiny Baseline: 88.42				**DeiT-Small** Baseline: 92.62				**DeiT-Base** Baseline: 93.64			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
Random Drop	87.94	84.02	75.22	33.18	92.06	90.52	85.00	50.34	92.68	90.62	82.14	25.86
EViT	88.44	86.42	80.34	52.54	**92.46**	91.80	89.02	70.16	**93.76**	93.06	89.00	51.32
ToMe	88.44	86.70	82.92	–	92.42	91.54	89.96	–	93.26	92.00	85.30	–
Top-K	**88.48**	86.28	80.14	52.02	**92.46**	91.60	89.16	68.78	93.74	**93.18**	88.86	**53.04**
Hybrid (Ours)	88.06	**86.96**	**84.00**	**67.62**	92.30	**91.86**	**90.60**	**79.58**	93.56	92.58	**89.72**	47.88

	ViT-Tiny Baseline: 89.22				**ViT-Small** Baseline: 93.18				**ViT-Base** Baseline: 95.52			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
Random Drop	63.42	57.04	42.22	13.20	88.62	83.82	67.48	19.16	92.08	90.04	80.66	33.90
EViT	**89.08**	86.92	79.72	44.06	93.06	91.28	82.80	35.48	95.54	95.04	92.48	61.00
ToMe	88.76	86.86	83.06	–	92.76	91.68	86.58	–	95.30	94.64	91.42	–
Top-K	**89.08**	86.78	79.22	42.90	**93.14**	91.22	81.82	34.44	**95.58**	**95.10**	92.46	58.72
Hybrid (Ours)	88.90	**87.62**	**84.14**	**60.12**	92.88	**91.94**	**88.14**	**55.58**	95.36	95.20	**93.62**	**71.16**

(ImageNet-1K subset) using semantic-preserving transformations. Our *Prune-and-Merge* blocks are integrated into the architecture at layers $l \in \{3, 6, 9\}$, specifically positioned post-attention and pre-MLP. In Stage 1, the patch importance scores are derived by averaging attention weights across all H heads from the `[CLS]` token [7]. For Stage 2 and all ToMe-based baselines, we utilize proportional attention [2] to maintain mathematical consistency during self-attention.

We benchmark our method against a comprehensive set of baselines to establish performance bounds:

- **Raw Model:** The original, uncompressed backbone operating on the full sequence length.
- **Random Drop:** A naive baseline where tokens are discarded uniformly at random, representing a lower bound for pruning.
- **Top-K Pruning:** A basic pruning strategy that retains high-attention tokens but discards the remainder without context fusion.
- **EViT & ToMe:** The state-of-the-art in semantic pruning [7] and similarity-based merging [2], respectively.

All reduction methods are evaluated across a spectrum of composite keep-rates $K \in \{0.25, 0.5, 0.7, 0.9\}$. Performance is quantified using Top-1 accuracy, computational complexity (GFLOPs), inference latency (ms/img) and throughput (img/sec). For a fair comparison, all baselines are evaluated under identical hardware and software constraints.

4.2. Results and Discussion

We evaluate the proposed *Prune-and-Merge* framework against four baselines across a spectrum of keep-rates K. Our results, summarized in Tables 1 and 3, demonstrate that the hybrid approach consistently achieves a superior top-1 accuracy compared to standalone reduction methods.

Table 2. Balanced vs pruning-heavy vs merging-heavy hybrid strategies (Accuracy %)

	DeiT-Tiny				DeiT-Small				DeiT-Base			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
Balanced	88.06	**86.96**	84.00	67.62	**92.30**	91.86	**90.60**	79.58	**93.56**	92.58	89.72	47.88
Pruning-heavy	**88.28**	86.60	82.66	68.00	92.26	91.96	90.40	79.76	93.44	**92.96**	**90.24**	52.44
Merging-heavy	88.00	86.88	**84.38**	**74.98**	92.28	**92.06**	90.26	**84.84**	93.46	92.70	88.70	**68.58**

	ViT-Tiny				ViT-Small				ViT-Base			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
Balanced	88.90	87.62	84.14	60.12	**92.88**	91.94	88.14	55.58	95.36	**95.20**	**93.62**	71.16
Pruning-heavy	**89.30**	87.52	82.56	60.06	92.86	92.02	87.58	55.98	**95.48**	95.20	93.62	74.08
Merging-heavy	89.10	**87.74**	**84.36**	**71.42**	92.84	**92.24**	**88.50**	**71.16**	95.28	95.16	93.12	**83.56**

Table 3. GFLOPs comparison for DeiT and ViT models under different keep rates.

	DeiT-Tiny Baseline: 1.079				DeiT-Small Baseline: 4.250				DeiT-Base Baseline: 16.867			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
EViT	0.989	0.752	0.576	0.432	3.817	2.892	2.203	1.634	14.985	11.336	8.608	6.349
ToMe	0.983	0.746	0.571	–	3.787	2.863	2.178	–	14.865	11.216	8.507	–
Top-K	**0.981**	**0.744**	**0.570**	**0.428**	**3.786**	**2.862**	**2.177**	**1.618**	**14.863**	**11.215**	**8.506**	**6.287**
Hybrid (Ours)	0.992	0.753	0.577	0.429	3.823	2.893	2.204	1.624	15.005	11.337	8.608	6.309

	ViT-Tiny Baseline: 1.079				ViT-Small Baseline: 4.250				ViT-Base Baseline: 16.867			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
EViT	0.989	0.752	0.576	0.432	3.817	2.892	2.203	1.634	14.985	11.336	8.608	6.349
ToMe	0.983	0.746	0.571	–	3.787	2.863	2.178	–	14.865	11.216	8.507	–
Top-K	**0.981**	**0.744**	**0.570**	**0.428**	**3.786**	**2.862**	**2.177**	**1.618**	**14.863**	**11.215**	**8.506**	**6.287**
Hybrid (Ours)	0.992	0.753	0.577	0.429	3.823	2.893	2.204	1.624	15.005	11.337	8.608	6.309

Accuracy-Efficiency Trade-offs: While our framework maintains a computational profile (GFLOPs and throughput) competitive with state-of-the-art methods such as EViT and ToMe, it exhibits significantly higher Top-1 accuracy across all architectural scales. This performance gap is most pronounced in aggressive compression regimes (e.g., $K = 0.25$), representing a marginal but consistent improvement over standalone techniques. This suggests that the interplay between pruning and merging effectively preserves the representational power of the transformer backbones even under significant sequence reduction.

Controlled Aggression Strategy: A primary contribution of this work is the introduction of Controlled Aggression within the reduction pipeline. Unlike fixed-strategy baselines, our framework allows for the dynamic balancing of pruning and merging stages. We observe that in lower keep-rates, prioritizing *aggressive merging* over pruning yields higher representational stability. As detailed in Table 2, by strategically shifting the reduction burden from Stage 1 (noise removal) to Stage 2 (information fusion) as K decreases, we mitigate the 'cliff-edge' accuracy degradation typically observed in pure-pruning baselines. This flexibility ensures that critical semantic tokens are fused rather than discarded, maintaining a more coherent global context.

4.3. Stability and Robustness Analysis

To quantify the structural reliability of the Prune-and-Merge framework, we evaluate its performance through two specialized metrics: Prediction Flip Rate (PFR) and Relative Accuracy Drop (RAD). These metrics allow us to distinguish between raw classification accuracy and the underlying stability of the model's decision-making process.We define RAD as:

$$RAD = \frac{\mathrm{Acc}(Y, T(x))}{\mathrm{Acc}(Y, x)} - \frac{\mathrm{Acc}(X, T(x))}{\mathrm{Acc}(X, x)} \tag{8}$$

where X and Y represent different frameworks, x is the input image and T is a semantic-preserving transformation. RAD serves as an accuracy trade-off metric, measuring the difference in top-1 accuracy ratios between frameworks when subjected to transformations.We define PFR as:

$$PFR = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[f(x_i) \neq f(T(x_i))] \tag{9}$$

where N is the number of samples and $\mathbb{I}$ is the indicator function. We utilize these metrics to quantify 'hallucinations' (H) within our framework. In this context, a hallucination represents a lack of faithfulness between a model's representation and its prediction. While faithfulness is often tested by removing important tokens, we evaluate the inverse: a robust model should maintain its prediction if an input is slightly transformed such that its semantic information remains unchanged. Thus, we define H as the deviation in flip rates:

$$H = \Delta PFR = |PFR_{\text{raw}} - PFR_{\text{hybrid}}| \tag{10}$$

where PFR_{raw} is the baseline flip rate of the backbone without token reduction. A ΔPFR value closer to zero indicates higher robustness against hallucinations.

Robustness to Hallucinations: We benchmark the ΔPFR (Table 4) across three augmentations: Rotation, Color Jitter and Horizontal Flips. We calculate this metric for ViT-Tiny and ViT-Base to capture trends across increasing model scales. We observe that ViT-Base exhibits significantly higher stability than ViT-Tiny under aggressive compression ($K = 0.25$). This suggests that the increased parameter density of the Base model provides more stable 'anchor' tokens, allowing the Prune-and-Merge stages to operate without disrupting the core decision boundary.

Manifold Integrity via RAD: The RAD analysis provides a structural comparison between our framework and existing paradigms. A value of $RAD \approx 0$ suggests that the framework hallucinates no more than the baseline it is compared to, whereas a positive value indicates higher hallucination rates. As shown in Table 5, RAD values remain near zero or slightly positive when compared with the raw ViT, signifying that the hybrid model preserves semantic faithfulness relative to the pretrained backbone. Furthermore, negative RAD values in comparison with other reduction frameworks indicate that our model exhibits lower hallucination rates than those baselines.

Stability at the Edge: Notably, the ToMe vs. Hybrid comparison yields NaN values at $K = 0.25$ because standalone merging cannot exceed a 50% reduction rate while maintaining a complete bipartite graph for tokens. In contrast, our proposed framework maintains stable RAD scores across the entire spectrum of K. This validates the utility of Stage 1 pruning in stabilizing the token set and removing noise before information fusion occurs in Stage 2.

Table 4. Average ΔPFR across augmentations.

Model	0.25	0.50	0.70	0.90
ViT-Base	0.3611	0.0422	0.0099	0.0022
ViT-Tiny	0.4105	0.0943	0.0272	0.0059

Table 5. Relative Accuracy Drop (RAD).

Model	Comparison	0.25	0.50	0.70	0.90
Tiny	Raw	0.073	0.023	0.003	-0.004
	ToMe	–	-0.003	-0.003	-0.003
	EViT	-0.030	-0.002	0.000	-0.003
Base	Raw	0.045	0.009	0.001	-0.001
	ToMe	–	0.002	-0.001	-0.001
	EViT	-0.026	0.000	0.001	-0.002

5. Conclusion

In this work, we introduced *Prune-and-Merge ViT*, a dual-stage token reduction framework designed to optimize the trade-off between inference efficiency and representational faithfulness. By implementing a 'Controlled Aggression' strategy, our method strategically transitions from top-k pruning in early stages to bipartite matching for information fusion in deeper layers.

Our empirical evaluation on ImageNet-100 demonstrates that *Prune-and-Merge ViT* significantly outperforms standalone pruning and merging baselines. Specifically, we achieve a 3× speedup on ViT-Base models at aggressive keep rates while maintaining superior accuracy. Furthermore, our stability analysis via RAD and PFR metrics confirms that the hybrid approach mitigates model hallucinations and maintains a more robust attention manifold compared to fixed-strategy alternatives. These results validate that a hardware-aware, hybrid reduction paradigm is essential for scaling Vision Transformers to resource-constrained environments without sacrificing reliability.

References

[1] Daniel Bolya and Judy Hoffman. Token merging for fast stable diffusion, 2023. 2

[2] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster, 2023. 1, 2, 3, 4

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. 1, 2

[4] Omar Erak, Omar Alhussein, Hatem Abou-Zeid, Mehdi Bennis, and Sami Muhaidat. Adaptive token merging for efficient transformer semantic communication at the edge, 2025. 2

[5] Joakim Bruslund Haurum, Sergio Escalera, Graham W. Taylor, and Thomas B. Moeslund. Which tokens to use? investigating token reduction in vision transformers, 2023. 2, 3

[6] Shibo Jie, Yehui Tang, Jianyuan Guo, Zhi-Hong Deng, Kai Han, and Yunhe Wang. Token compensator: Altering inference cost of vision transformer without re-tuning, 2024. 2

[7] Youwei Liang, Chongjian Ge, Zhan Tong, Yibing Song, Jue Wang, and Pengtao Xie. Not all patches are what you need: Expediting vision transformers via token reorganizations, 2022. 1, 2, 3, 4

[8] Junzhu Mao, Yang Shen, Jinyang Guo, Yazhou Yao, and Xiansheng Hua. Efficient token compression for vision transformer with spatial information preserved, 2025. 2

[9] Liu Peng and Yaochu Jin. Robustness of probabilistic models to low-quality data: A multi-perspective analysis, 2025. 2

[10] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification, 2021. 2

[11] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention, 2021. 1, 2

[12] Guangzhen Yao, Jiayun Zheng, Zezhou Wang, Wenxin Zhang, Renda Han, Chuangxin Zhao, Zeyu Zhang, and Runhao Liu. V-pruner: A fast and globally-informed token pruning framework for vision transformer. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(40):34396–34404, 2026. 2

[13] Hanxun Yu, Wentong Li, Xuan Qu, Song Wang, Junbo Chen, and Jianke Zhu. Visiontrim: Unified vision token compression for training-free mllm acceleration, 2026. 2

[14] Zheng Zhan, Zhenglun Kong, Yifan Gong, Yushu Wu, Zichong Meng, Hangyu Zheng, Xuan Shen, Stratis Ioannidis, Wei Niu, Pu Zhao, and Yanzhi Wang. Exploring token pruning in vision state space models, 2024. 2

[15] Jusheng Zhang, Xiaoyang Guo, Kaitong Cai, Qinhan Lv, Yijia Fan, Wenhao Chai, Jian Wang, and Keze Wang. Hybridtoken-vlm: Hybrid token compression for vision-language models, 2025. 2

Appendix

In this appendix, we provide additional technical details regarding the hardware-specific performance of our framework and the experimental setup used to generate the results presented in the main text.

A. Extended Hardware Performance Metrics

While the main text focuses on accuracy and GFLOPs, we present here the comprehensive latency and throughput benchmarks conducted on an T4 GPU using a Google Colab environment.

A.1. Inference Latency

Table 6 details the average per-image inference latency for both DeiT and ViT architectures. We observe that the dual-stage overhead of our hybrid approach is negligible, consistently maintaining parity with, or outperforming, standalone pruning and merging methods. Notably, in the ViT-Base configuration at aggressive keep rates ($K = 0.25$), our hybrid method achieves the lowest latency of 4.222 ms/img, outperforming EViT (5.061 ms/img) and Top-K (4.417 ms/img). Similarly, for ViT-Tiny at $K = 0.9$, our approach achieves a state-of-the-art latency of 0.971 ms/img. These results demonstrate that the combination of pruning and merging stages does not introduce a computational bottleneck, even when scaling to larger backbone architectures.

Table 6. Latency comparison (ms/img)

	DeiT-Tiny Baseline: 1.629				**DeiT-Small** Baseline: 3.092				**DeiT-Base** Baseline: 12.772			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
EViT	1.241	0.984	0.751	0.747	3.256	2.390	1.763	1.253	11.559	8.771	6.728	5.058
ToMe	**1.042**	**0.888**	0.900	–	2.952	2.217	1.621	–	10.437	7.958	6.092	–
Top-K	1.156	0.903	**0.729**	**0.633**	**2.892**	**2.138**	**1.569**	**1.311**	**10.107**	**7.660**	**5.845**	**4.421**
Hybrid (Ours)	1.110	1.116	1.249	1.107	2.930	2.165	1.628	1.193	10.189	7.626	5.885	4.532

	ViT-Tiny Baseline: 1.322				**ViT-Small** Baseline: 3.440				**ViT-Base** Baseline: 12.721			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
EViT	1.096	1.036	**0.753**	**0.660**	3.260	2.362	1.744	1.254	11.568	8.815	6.698	5.061
ToMe	1.059	1.031	0.777	–	2.745	2.203	1.647	–	10.301	7.361	6.022	–
Top-K	1.208	0.901	**0.751**	0.719	2.876	2.136	1.575	**1.150**	10.114	7.646	5.825	4.417
Hybrid (Ours)	**0.971**	**0.932**	1.305	0.697	**2.822**	**2.093**	**1.560**	1.227	**9.805**	**7.340**	**5.631**	**4.222**

A.2. System Throughput

Table 7 presents a comparative analysis of system throughput (images/sec) across varied keep rates for DeiT and ViT architectures. Our Hybrid framework demonstrates superior scaling characteristics, particularly as model complexity increases from Tiny to Base variants.In the ViT-Base regime, our method consistently achieves the highest throughput across almost all keep rates. Most notably, at the most aggressive compression level ($K = 0.25$), the Hybrid approach reaches 236.85 imgs/sec, representing a 3.01× speedup over the ViT-Base baseline (78.61 imgs/sec) and significantly outperforming EViT (197.60 imgs/sec). Even at higher keep rates such as $K = 0.9$, our method maintains a lead with 101.98 imgs/sec, validating that the dual-stage reduction logic provides an immediate efficiency boost without requiring a heavy computational overhead. For smaller scales like ViT-Tiny, our method achieves a peak throughput of 1073.18 imgs/sec at $K = 0.7$, further illustrating its versatility for high-speed edge deployment.

B. Supplemental Implementation Details

This section expands upon the implementation details provided in Section 4. We provide specific hyperparameter configurations and environmental constraints to ensure full reproducibility.

Table 7. Throughput comparison (imgs/sec)

	DeiT-Tiny Baseline: 613.92				**DeiT-Small** Baseline: 323.46				**DeiT-Base** Baseline: 78.30			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
EViT	805.73	1016.41	1330.76	1338.06	307.11	418.34	567.28	797.83	86.51	114.02	148.62	197.69
ToMe	**960.08**	**1126.47**	1110.56	–	338.74	451.12	616.72	–	95.81	125.66	164.14	–
Top-K	864.74	1107.43	**1371.64**	**1579.34**	**345.74**	**467.62**	**637.31**	762.66	98.94	130.56	**171.08**	**226.19**
Hybrid (Ours)	917.30	895.66	800.48	903.08	341.34	461.98	614.11	**838.28**	**98.14**	**131.13**	169.92	220.63

	ViT-Tiny Baseline: 756.22				**ViT-Small** Baseline: 290.71				**ViT-Base** Baseline: 78.61			
Keep Rates	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25	0.9	0.7	0.5	0.25
EViT	912.27	965.17	1328.21	**1514.27**	306.75	423.41	573.48	797.14	86.45	113.44	149.30	197.60
ToMe	944.52	969.48	1287.70	–	**364.32**	454.01	607.31	–	97.07	135.85	166.05	–
Top-K	827.67	1109.70	**1331.42**	1390.24	347.75	468.19	634.78	**869.25**	98.87	130.79	171.68	226.38
Hybrid (Ours)	**1029.53**	**1073.18**	766.50	1434.45	354.34	**477.73**	**640.87**	815.07	**101.98**	**136.25**	**177.57**	**236.85**

Hyperparameter Configuration All experiments were conducted using the `PyTorch` framework and the `timm` library on a Google Colab T4 GPU environment. For our hybrid Prune-and-Merge blocks, the similarity metric used for token merging in Stage 2 was the *cosine similarity* computed on the token features (embeddings)

Hardware Environment All benchmarks (Latency in Table 6 and Throughput in Table 7) were measured on a single NVIDIA T4 GPU (16GB VRAM).

ImageNet-100 Class Mapping The 100 classes were selected as a contiguous subset of the ImageNet-1K labels. While the backbone features were initialized from ImageNet-1K pre-training/finetuning, only the final linear projection layer was modified to match the 100-class dimensionality.

C. Additional Results

In this section, we provide detailed quantitative results supporting our analysis. We report (i) Prediction Flip Rate (PFR) and (ii) absolute change in PFR (ΔPFR)across different augmentations and keep rates.

C.1. Prediction Flip Rate (PFR)

Table 8. Prediction Flip Rate (PFR) for raw and hybrid models across augmentations and keep rates.

Model	Augmentation	Raw	0.25	0.50	0.70	0.90
ViT-Base	Rotation	0.0472	0.4528	0.0938	0.0552	0.0458
	Color Jitter	0.0128	0.3552	0.0524	0.0250	0.0162
	Horizontal Flip	0.0112	0.3464	0.0516	0.0206	0.0130
ViT-Tiny	Rotation	0.1856	0.6086	0.2902	0.2162	0.1898
	Color Jitter	0.0416	0.4516	0.1394	0.0698	0.0498
	Horizontal Flip	0.0356	0.4342	0.1160	0.0584	0.0408

C.2. Absolute Change in PFR (ΔPFR)

CVPR
#*****

Table 9. Absolute change in Prediction Flip Rate (ΔPFR) between raw and hybrid models.

Model	Augmentation	0.25	0.50	0.70	0.90
ViT-Base	Rotation	0.4056	0.0466	0.0080	0.0014
	Color Jitter	0.3424	0.0396	0.0122	0.0034
	Horizontal Flip	0.3352	0.0404	0.0094	0.0018
ViT-Tiny	Rotation	0.4230	0.1046	0.0306	0.0042
	Color Jitter	0.4100	0.0978	0.0282	0.0082
	Horizontal Flip	0.3986	0.0804	0.0228	0.0052